Namaste JavaScript — Episode 1 Study Guide

Topic: How JavaScript Works & Execution Context

Speaker: Akshay Saini

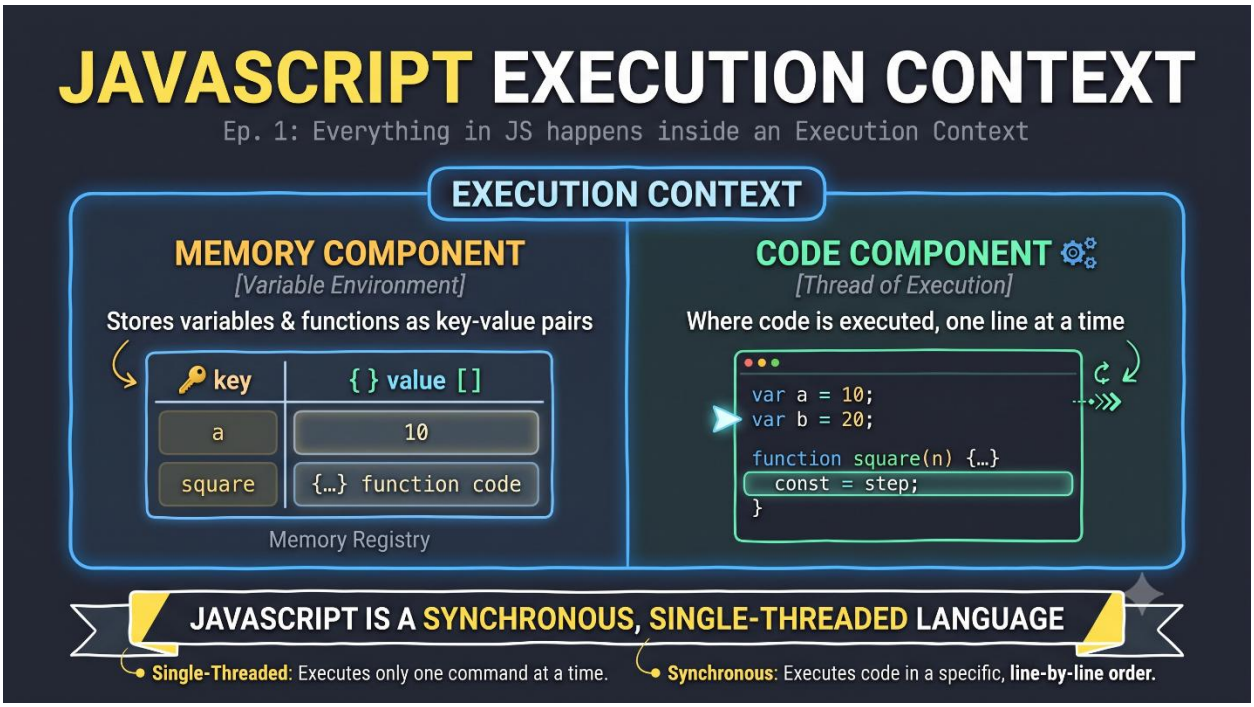
1. The Core Principle

"Everything in JavaScript happens inside an Execution Context."

Whenever JavaScript code is executed, the JavaScript engine wraps the runtime environment inside an abstract container called the **Execution Context**. No code runs outside of one.

2. Structure of the Execution Context

An Execution Context consists of two primary components operating side-by-side:



Component A: Memory Component (Variable Environment)

- **Role:** Stores variables and function definitions.
- **Format:** Key-value pairs (key : value).
    - Variables store an identifier and assigned value (e.g., a : 10).
    - Functions store the function identifier and the complete function body.
- **Formal Name:** Variable Environment.

Component B: Code Component (Thread of Execution)

- **Role:** Evaluates computations, assignments, and expressions.
- **Execution Pattern:** Sequentially, **one line at a time**.
- **Formal Name:** Thread of Execution.

3. Runtime Nature of JavaScript

JavaScript is defined as a **synchronous, single-threaded language**.

Attribute	Definition	Practical Implication
Single-Threaded	JavaScript has a single thread of execution.	It can only execute **one command at a time**. Parallel execution within the engine core does not occur.
Synchronous	Instructions run in strict sequential order.	It moves to line $N + 1$ only after line N has finished executing.

4. Key Terminology Quick Reference

- **Execution Context:** The wrapper/environment created by the engine to evaluate and execute JavaScript programs.
- **Variable Environment:** The formal specification term for the memory allocation component of an execution context.
- **Thread of Execution:** The sequential pipeline in which instructions are read, dispatched, and evaluated.
- **Synchronous Single-Threaded:** Processes one command at a time in a deterministic, linear order.

5. Preview of Execution Lifecycle

The complete lifecycle of any Execution Context involves two discrete phases:

1. **Memory Creation Phase:** Allocates memory addresses for variables (undefined) and functions (entire definition body).
2. **Code Execution Phase:** Evaluates expressions line by line and assigns literal values to memory slots.

Here is the complete printable Markdown study guide for **Episode 2 of Namaste JavaScript: "How JavaScript Code is Executed? & Call Stack"** by Akshay Saini.

Namaste JavaScript — Episode 2 Study Guide

Topic: How JavaScript Code is Executed & The Call Stack

Speaker: Akshay Saini

1. Core Principle

- Whenever a JavaScript program runs, a **Global Execution Context (GEC)** is created and pushed onto the Call Stack.
- Every Execution Context is constructed in **two distinct phases**:
    1. **Phase 1: Memory Creation Phase**
    2. **Phase 2: Code Execution Phase**

2. Walkthrough Code Example

JavaScript

```
var n = 2;

function square(num) {
   var ans = num * num;
   return ans;
}

var square2 = square(n);

var square4 = square(4);
```

3. The Two Phases in Detail

Phase 1: Memory Creation Phase

Before running any code, JavaScript scans through the script line by line and allocates memory space for identifiers:

- **Variables (var):** Allocated memory slots and initialized with the special keyword/placeholder **undefined**.
    -

    - n : undefined
    - square2 : undefined
    - square4 : undefined
- **Functions:** The complete function definition code is stored in the memory component.
    - square : function(num) { ... }

Phase 2: Code Execution Phase

JavaScript executes the code sequentially, one line at a time:

1. **var n = 2;**

The literal value 2 replaces undefined in memory for n.

2. **function square(num) { ... }**

Skipped during the execution pass because the definition was already loaded into memory during Phase 1.

3. **var square2 = square(n);**

A **function invocation** is encountered (()).

    - A **brand new Function Execution Context** is created.
    - It is pushed onto the Call Stack.

4. Function Execution Context Lifecycle

Inside square(n):

1. **Local Memory Allocation Phase:**
    - Parameters and local variables are allocated with undefined:
        - num : undefined
        - ans : undefined
2. **Local Code Execution Phase:**
    - Argument n (2) is assigned to the parameter num (num = 2).
    - Expression num * num evaluates to 4, replacing undefined for ans.
    - **return ans;:** Signals that execution is finished and passes value 4 back to where the function was called (square2 = 4).

3. **Destruction:**
    - The function's execution context is **completely deleted** and popped off the Call Stack.
    - Control returns to the Global Execution Context.

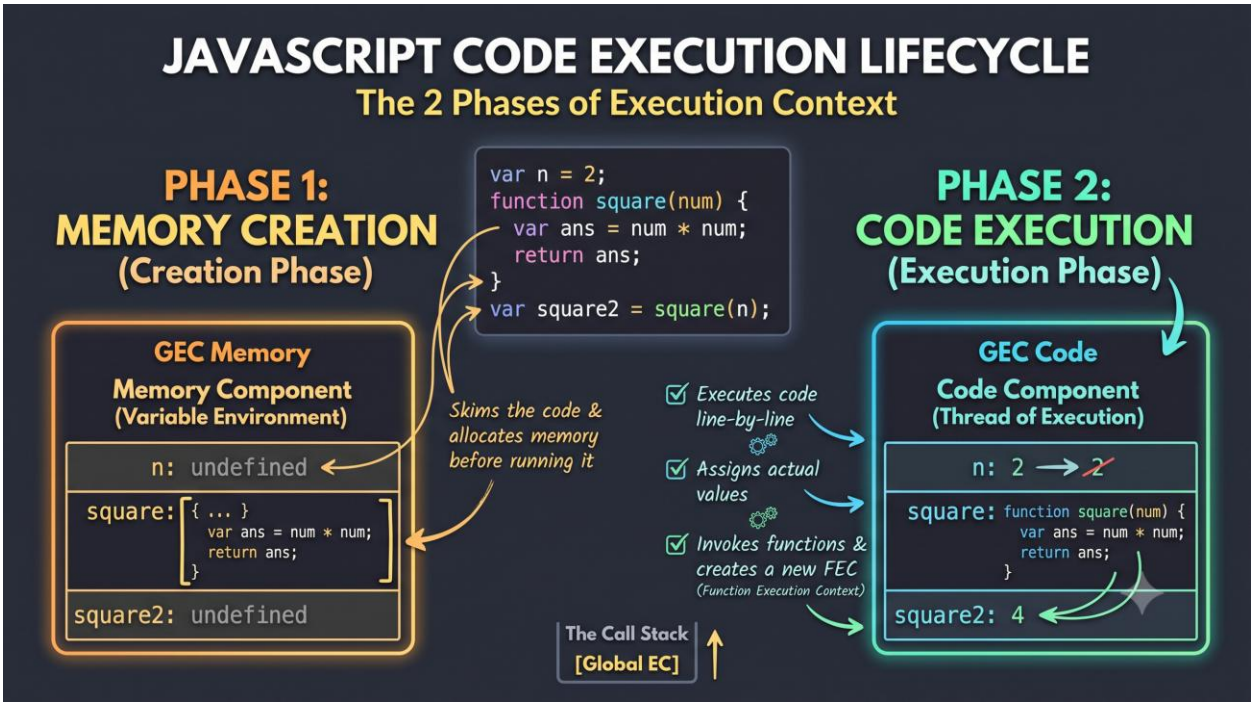
(The exact same lifecycle repeats when square(4) is invoked on the next line).

5. The Call Stack

The Call Stack is a data structure used by the JavaScript engine to manage and track execution contexts.

Plaintext

Call Stack Lifecycle:



Key Responsibilities:

- **Base Layer:** The Global Execution Context resides permanently at the bottom until the program finishes.
- **Order of Execution:** The Call Stack maintains and enforces the order in which execution contexts run and terminate.
- **Cleanup:** Once all code has finished running, the Global Execution Context is popped, leaving the stack completely empty.

Alternative Names:

In specifications and developer documentation, the Call Stack is also known as:

- **Execution Context Stack**
- **Program Stack**
- **Control Stack**
- **Runtime Stack**
- **Machine Stack**